

IM System Design Document

1. 文档信息

项目	内容
文档名称	IM 系统设计文档
技术栈	Spring Boot WebFlux + Spring Data JPA + PostgreSQL
当前阶段	模块化单体
后续演进	微服务化、Redis 路由、MQ 投递、MinIO 文件系统
核心目标	支持私聊、群聊、频道、消息投递、会话列表、断线同步、高并发扩展

2. 设计目标

本系统目标是实现一个可演进的 IM 内核，当前以单体架构落地，但在模型设计、模块边界、消息投递机制上预留未来微服务化能力。

2.1 当前目标

1. 支持私聊、群聊、频道三类消息场景。
2. 支持 WebSocket 长连接。
3. 支持消息可靠落库。
4. 支持服务端 ACK。
5. 支持离线消息同步。
6. 支持用户会话列表。
7. 支持小群写扩散、大群读扩散。
8. 支持消息幂等。
9. 支持 target 内消息顺序。
10. 支持文件元数据预留，后续接入 MinIO。

2.2 非目标

当前阶段暂不实现：

1. 多机 WebSocket 路由。
2. 完整离线移动推送。
3. 消息全文搜索。
4. 端到端加密。
5. 超大群完整精确未读数。
6. 多活容灾。

7. 完整音视频通话。

3. 核心设计原则

3.1 消息发送和消息投递解耦

消息发送链路只负责：

1. 鉴权。
2. 权限校验。
3. 幂等校验。
4. 分配消息序号。
5. 消息落库。
6. 写入 Outbox 事件。

消息投递链路异步执行：

1. 更新用户会话列表。
2. 在线 WebSocket 推送。
3. 离线同步准备。
4. 投递失败重试。
5. 后续审计、搜索索引、机器人事件扩展。

3.2 消息只存一份

私聊、群聊、频道消息统一存储在 `im_message` 表中。

不要为每个用户复制一份消息。

用户看到的“会话列表”由 `im_user_inbox` 表维护。

3.3 Conversation 概念拆分

系统不使用一个混乱的 `Conversation` 实体承载所有概念。

本系统将其拆成：

概念	说明
ChatTarget	统一消息投递目标
DirectChat	私聊目标
GroupChat	群聊目标
Channel	频道目标

概念	说明
UserInbox	用户侧会话列表投影
Thread	子话题、楼中楼、回复线

3.4 只保证 target 内有序

系统不保证全局消息有序。

只保证：

```
target_id + seq 有序
```

客户端按照 `targetId + lastSeq` 进行断线同步。

3.5 WebFlux 不直接执行 JPA 阻塞操作

WebFlux 使用 Netty event loop 处理网络 I/O。

JPA 是阻塞式 ORM，必须放到专门的 JPA Scheduler 中执行。

推荐方式：

```
Mono.fromCallable(() -> txService.sendInTransaction(command))
    .subscribeOn(jpaScheduler);
```

禁止在 Netty event loop 中直接调用 JPA Repository。

4. 总体架构

4.1 当前架构：模块化单体

```
im-monolith
├── im-gateway      WebSocket 长连接、鉴权、心跳、上下行帧
├── im-message      消息发送、消息落库、seq 分配、幂等
├── im-delivery      Outbox 投递、在线推送、离线同步
├── im-target        ChatTarget / DirectChat / GroupChat / Channel
├── im-member        成员关系、权限、禁言、角色
├── im-inbox         用户会话列表、未读数、lastReadSeq
├── im-thread        Thread / Topic / Reply Line
├── im-file          文件元数据，后续接 MinIO
```

└─ im-user	用户、设备、在线状态
└─ im-common	ID、事件、异常、限流、审计

4.2 模块调用关系

```

flowchart TD
    Client[Client] --> WS[WebSocket Handler]
    WS --> Router[Inbound Command Router]
    Router --> App[Application Service]
    App --> JpaWrap[Mono.fromCallable + JPA Scheduler]
    JpaWrap --> Tx[Blocking Transactional Service]
    Tx --> Validator[Validation Chain]
    Validator --> Seq[Message Seq Service]
    Seq --> MsgRepo[Message Repository]
    Tx --> OutboxRepo[Outbox Repository]
    OutboxRepo --> Dispatcher[Outbox Dispatcher]
    Dispatcher --> Delivery[Message Delivery Service]
    Delivery --> Strategy[Delivery Strategy Router]
    Strategy --> Inbox[Inbox Updater]
    Strategy --> Conn[Connection Manager]
    Conn --> WS
  
```

5. 技术选型

技术	用途	说明
Spring Boot	应用基础框架	统一配置、Bean 管理、监控
Spring WebFlux	HTTP API、WebSocket	非阻塞网络层
Spring Data JPA	数据访问	阻塞式 ORM，需要隔离线程池
PostgreSQL	主数据库	存储消息、会话、成员、Outbox
Flyway	数据库版本管理	所有 DDL 通过版本脚本管理
Reactor	异步编排	Mono / Flux
HikariCP	JDBC 连接池	控制数据库连接数
Redis	后续扩展	多节点路由、分布式限流、在线状态

技术	用途	说明
MQ	后续扩展	Kafka / RabbitMQ，用于替代本地 Outbox Dispatcher
MinIO	后续扩展	文件对象存储

6. 领域模型

6.1 核心实体

实体	说明
User	用户
DeviceSession	用户设备连接
ChatTarget	统一消息目标
DirectChat	私聊
GroupChat	群聊
Channel	频道
TargetMember	成员关系
Message	消息
MessageSeq	target 内消息序号
UserInbox	用户会话列表
OutboxEvent	可靠事件
Thread	消息子话题
Attachment	文件元数据

6.2 领域关系

```
classDiagram
    class User {
        Long id
        String nickname
    }

    class ChatTarget {
        Long id
        Long tenantId
    }
```

```

        TargetType type
        TargetStatus status
    }

    class DirectChat {
        Long targetId
        Long userAId
        Long userBId
        String pairKey
    }

    class GroupChat {
        Long targetId
        String name
        Long ownerId
        Integer memberCount
    }

    class Channel {
        Long targetId
        String name
        ChannelMode mode
        ChannelVisibility visibility
    }

    class TargetMember {
        Long targetId
        Long userId
        MemberRole role
        Long lastReadSeq
        Boolean muted
    }

    class Message {
        Long id
        Long targetId
        Long seq
        Long senderId
        String clientMsgId
        MessageType contentType
        String content
    }

    class UserInbox {
        Long userId
        Long targetId
        Long lastMessageId
        Long lastSeq
    }

```

```

        Long lastReadSeq
        Integer unreadCount
    }

    class OutboxEvent {
        Long id
        String eventType
        String payload
        String status
    }

    ChatTarget "1" --> "0..1" DirectChat
    ChatTarget "1" --> "0..1" GroupChat
    ChatTarget "1" --> "0..1" Channel
    ChatTarget "1" --> "many" TargetMember
    ChatTarget "1" --> "many" Message
    User "1" --> "many" TargetMember
    User "1" --> "many" UserInbox
    ChatTarget "1" --> "many" UserInbox

```

7. 数据库设计

7.1 im_chat_target

统一消息目标表。

```

CREATE TABLE im_chat_target (
    id BIGINT PRIMARY KEY,
    tenant_id BIGINT NOT NULL,
    type VARCHAR(32) NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_by BIGINT NOT NULL,
    created_at TIMESTAMP(6) NOT NULL,
    updated_at TIMESTAMP(6) NOT NULL,
    deleted BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE INDEX idx_chat_target_tenant_type
ON im_chat_target (tenant_id, type);

```

7.2 im_direct_chat

私聊表。

```
CREATE TABLE im_direct_chat (
  target_id BIGINT PRIMARY KEY,
  user_a_id BIGINT NOT NULL,
  user_b_id BIGINT NOT NULL,
  pair_key VARCHAR(128) NOT NULL,
  created_at TIMESTAMP(6) NOT NULL,

  CONSTRAINT uk_direct_pair UNIQUE (pair_key)
);
```

`pair_key` 生成规则：

```
min(userAId, userBId) + ":" + max(userAId, userBId)
```

7.3 im_group_chat

群聊表。

```
CREATE TABLE im_group_chat (
  target_id BIGINT PRIMARY KEY,
  name VARCHAR(128) NOT NULL,
  owner_id BIGINT NOT NULL,
  avatar_url VARCHAR(512),
  member_count INT NOT NULL DEFAULT 0,
  group_type VARCHAR(32) NOT NULL,
  created_at TIMESTAMP(6) NOT NULL,
  updated_at TIMESTAMP(6) NOT NULL
);
```

7.4 im_channel

频道表。

```
CREATE TABLE im_channel (
  target_id BIGINT PRIMARY KEY,
  workspace_id BIGINT,
  name VARCHAR(128) NOT NULL,
  visibility VARCHAR(32) NOT NULL,
  mode VARCHAR(32) NOT NULL,
  created_by BIGINT NOT NULL,
  created_at TIMESTAMP(6) NOT NULL,
```



```
updated_at TIMESTAMP(6) NOT NULL
);
```

频道模式：

mode	说明
DISCUSSION	普通讨论频道
BROADCAST	广播频道，只允许管理员或机器人发言

7.5 im_target_member

成员关系表。

```
CREATE TABLE im_target_member (
  target_id BIGINT NOT NULL,
  user_id BIGINT NOT NULL,
  role VARCHAR(32) NOT NULL,
  status VARCHAR(32) NOT NULL,
  muted BOOLEAN NOT NULL DEFAULT FALSE,
  last_read_seq BIGINT NOT NULL DEFAULT 0,
  joined_at TIMESTAMP(6) NOT NULL,
  updated_at TIMESTAMP(6) NOT NULL,

  PRIMARY KEY (target_id, user_id)
);

CREATE INDEX idx_target_member_user
ON im_target_member (user_id, target_id);

CREATE INDEX idx_target_member_target
ON im_target_member (target_id, user_id);
```

7.6 im_message

消息表。

```
CREATE TABLE im_message (
  id BIGINT PRIMARY KEY,
  tenant_id BIGINT NOT NULL,
  target_id BIGINT NOT NULL,
  thread_id BIGINT,
  seq BIGINT NOT NULL,
  sender_id BIGINT NOT NULL,
```

```

client_msg_id VARCHAR(128) NOT NULL,
content_type VARCHAR(32) NOT NULL,
content JSONB NOT NULL,
status VARCHAR(32) NOT NULL,
created_at TIMESTAMP(6) NOT NULL,
updated_at TIMESTAMP(6) NOT NULL,
deleted BOOLEAN NOT NULL DEFAULT FALSE,

CONSTRAINT uk_message_target_seq UNIQUE (target_id, seq),
CONSTRAINT uk_message_sender_client UNIQUE (sender_id, client_msg_id)
);

CREATE INDEX idx_message_target_seq_desc
ON im_message (target_id, seq DESC);

CREATE INDEX idx_message_sender_created
ON im_message (sender_id, created_at DESC);

```

关键约束：

约束	作用
UNIQUE(target_id, seq)	保证 target 内消息顺序唯一
UNIQUE(sender_id, client_msg_id)	防止客户端重试导致重复消息

7.7 im_chat_target_seq

target 内消息序号表。

```

CREATE TABLE im_chat_target_seq (
    target_id BIGINT PRIMARY KEY,
    current_seq BIGINT NOT NULL
);

```

第一版序号生成 SQL：

```

INSERT INTO im_chat_target_seq (target_id, current_seq)
VALUES (:targetId, 1)
ON CONFLICT (target_id)
DO UPDATE SET current_seq = im_chat_target_seq.current_seq + 1
RETURNING current_seq;

```

7.8 im_user_inbox

用户会话列表表。

```
CREATE TABLE im_user_inbox (  
    user_id BIGINT NOT NULL,  
    target_id BIGINT NOT NULL,  
    last_message_id BIGINT,  
    last_seq BIGINT NOT NULL DEFAULT 0,  
    last_read_seq BIGINT NOT NULL DEFAULT 0,  
    unread_count INT NOT NULL DEFAULT 0,  
    pinned BOOLEAN NOT NULL DEFAULT FALSE,  
    muted BOOLEAN NOT NULL DEFAULT FALSE,  
    updated_at TIMESTAMP(6) NOT NULL,  
  
    PRIMARY KEY (user_id, target_id)  
);  
  
CREATE INDEX idx_user_inbox_updated  
ON im_user_inbox (user_id, updated_at DESC);
```

7.9 im_outbox_event

Outbox 事件表。

```
CREATE TABLE im_outbox_event (  
    id BIGINT PRIMARY KEY,  
    event_type VARCHAR(64) NOT NULL,  
    aggregate_id BIGINT NOT NULL,  
    payload JSONB NOT NULL,  
    status VARCHAR(32) NOT NULL,  
    retry_count INT NOT NULL DEFAULT 0,  
    next_retry_at TIMESTAMP(6),  
    locked_by VARCHAR(128),  
    locked_at TIMESTAMP(6),  
    created_at TIMESTAMP(6) NOT NULL,  
    updated_at TIMESTAMP(6) NOT NULL  
);  
  
CREATE INDEX idx_outbox_status_retry  
ON im_outbox_event (status, next_retry_at, id);
```

状态说明：

status	说明
PENDING	等待处理
PROCESSING	处理中
SUCCESS	处理成功
RETRY	等待重试
FAILED	最终失败

7.10 im_thread

Thread / Topic 表。

```
CREATE TABLE im_thread (
  id BIGINT PRIMARY KEY,
  target_id BIGINT NOT NULL,
  root_message_id BIGINT,
  title VARCHAR(255),
  created_by BIGINT NOT NULL,
  status VARCHAR(32) NOT NULL,
  created_at TIMESTAMP(6) NOT NULL,
  updated_at TIMESTAMP(6) NOT NULL
);

CREATE INDEX idx_thread_target
ON im_thread (target_id, updated_at DESC);
```

7.11 im_attachment

文件元数据表。

```
CREATE TABLE im_attachment (
  id BIGINT PRIMARY KEY,
  owner_id BIGINT NOT NULL,
  storage_type VARCHAR(32) NOT NULL,
  bucket VARCHAR(128),
  object_key VARCHAR(512),
  file_name VARCHAR(255) NOT NULL,
  content_type VARCHAR(128),
  size_bytes BIGINT NOT NULL,
  status VARCHAR(32) NOT NULL,
  created_at TIMESTAMP(6) NOT NULL
);
```

8. 消息协议设计

8.1 客户端发送消息

```
{
  "op": "SEND_MESSAGE",
  "requestId": "req-001",
  "body": {
    "clientMsgId": "client-uuid-001",
    "targetId": 10001,
    "threadId": null,
    "contentType": "TEXT",
    "content": {
      "text": "hello"
    }
  }
}
```

8.2 服务端发送 ACK

```
{
  "op": "SEND_ACK",
  "requestId": "req-001",
  "body": {
    "messageId": 90001,
    "targetId": 10001,
    "seq": 101,
    "serverTime": "2026-06-26T10:00:00"
  }
}
```

8.3 服务端推送消息

```
{
  "op": "MESSAGE_PUSH",
  "body": {
    "messageId": 90001,
    "targetId": 10001,
    "seq": 101,
    "senderId": 20001,
    "contentType": "TEXT",
    "content": {
      "text": "hello"
    }
  }
}
```

```
    },  
    "createdAt": "2026-06-26T10:00:00"  
  }  
}
```

8.4 客户端投递回执

```
{  
  "op": "DELIVERY_ACK",  
  "body": {  
    "messageId": 90001,  
    "targetId": 10001,  
    "seq": 101,  
    "deviceId": "macbook-001"  
  }  
}
```

8.5 客户端已读

```
{  
  "op": "READ_TARGET",  
  "body": {  
    "targetId": 10001,  
    "readSeq": 101  
  }  
}
```

9. HTTP API 设计

9.1 获取会话列表

```
GET /api/im/inboxes?cursor=xxx&limit=20
```

响应：

```
{  
  "items": [  
    {  
      "targetId": 10001,  
      "targetType": "DIRECT",  
      "title": "Mario",  
    }  
  ]  
}
```

```
    "lastMessageId": 90001,  
    "lastSeq": 101,  
    "lastReadSeq": 90,  
    "unreadCount": 11,  
    "pinned": false,  
    "muted": false,  
    "updatedAt": "2026-06-26T10:00:00"  
  }  
],  
  "nextCursor": "..."  
}
```

9.2 拉取消息

正向同步：

```
GET /api/im/targets/{targetId}/messages?afterSeq=100&limit=50
```

历史消息：

```
GET /api/im/targets/{targetId}/messages?beforeSeq=200&limit=50
```

响应：

```
{  
  "items": [  
    {  
      "messageId": 90001,  
      "targetId": 10001,  
      "seq": 101,  
      "senderId": 20001,  
      "contentType": "TEXT",  
      "content": {  
        "text": "hello"  
      },  
      "createdAt": "2026-06-26T10:00:00"  
    },  
  ],  
  "hasMore": true  
}
```

9.3 标记已读

```
POST /api/im/targets/{targetId}/read
```

请求：

```
{
  "readSeq": 120
}
```

9.4 创建私聊

```
POST /api/im/direct-chats
```

请求：

```
{
  "targetUserId": 20002
}
```

9.5 创建群聊

```
POST /api/im/groups
```

请求：

```
{
  "name": "技术讨论群",
  "memberIds": [20001, 20002, 20003]
}
```

9.6 创建频道

```
POST /api/im/channels
```

请求：


```
{
  "name": "系统公告",
  "mode": "BROADCAST",
  "visibility": "PRIVATE"
}
```

10. 核心业务流程

10.1 发送消息流程

```
sequenceDiagram
    participant C as Client
    participant W as WebSocketHandler
    participant A as SendMessageApplicationService
    participant T as SendMessageTxService
    participant DB as PostgreSQL
    participant O as Outbox
    participant D as OutboxDispatcher
    participant R as Receiver

    C->>W: SEND_MESSAGE
    W->>A: SendMessageCommand
    A->>T: Mono.fromCallable(...).subscribeOn(jpaScheduler)

    T->>T: 幂等校验 clientMsgId
    T->>T: 成员/禁言/权限/限流校验
    T->>DB: 生成 target seq
    T->>DB: 保存 message
    T->>O: 保存 MESSAGE_CREATED outbox
    T-->>A: SendAck
    A-->>W: SendAck
    W-->>C: SEND_ACK

    D->>O: 拉取 PENDING 事件
    D->>DB: 更新 inbox
    D->>R: 在线 WebSocket 推送
    D->>O: 标记 SUCCESS
```

10.2 断线重连同步流程

```
sequenceDiagram
    participant C as Client
```

```
participant S as Server
participant DB as MessageDB

C->>S: Reconnect with targetId + lastSeq
S->>DB: 查询 seq > lastSeq 的消息
DB-->>S: 返回缺失消息
S-->>C: SYNC_MESSAGES
C->>S: READ_TARGET / DELIVERY_ACK
```

10.3 小群消息投递

1. 消息落库。
2. 写入 OutboxEvent。
3. OutboxDispatcher 拉取事件。
4. 查询群成员。
5. 批量更新成员 UserInbox。
6. 对在线成员推送 WebSocket。
7. 离线成员下次打开时从 DB 拉取。

10.4 大群消息投递

1. 消息落库。
2. 写入 OutboxEvent。
3. OutboxDispatcher 拉取事件。
4. 不为全部成员更新 UserInbox。
5. 只推送当前在线的活跃成员。
6. 用户打开大群时根据 lastReadSeq 拉取消息。
7. 未读数通过 latestSeq - lastReadSeq 估算或展示 99+。

11. WebFlux + JPA 线程模型

11.1 原则

WebFlux 的 event loop 不能执行阻塞式 JPA 操作。

所有 JPA 调用必须进入专门的 Scheduler。

11.2 JPA Scheduler 配置

```
@Configuration
public class JpaSchedulerConfig {
```

```

@Bean(destroyMethod = "dispose")
public Scheduler jpaScheduler(
    @Value("${im.jpa.scheduler.thread-size:32}") int threadSize
) {
    return Schedulers.newBoundedElastic(
        threadSize,
        10000,
        "im-jpa"
    );
}
}

```

11.3 Application Service 写法

```

@Service
public class SendMessageApplicationService {

    private final Scheduler jpaScheduler;
    private final SendMessageTxService sendMessageTxService;

    public SendMessageApplicationService(
        Scheduler jpaScheduler,
        SendMessageTxService sendMessageTxService
    ) {
        this.jpaScheduler = jpaScheduler;
        this.sendMessageTxService = sendMessageTxService;
    }

    public Mono<SendMessageAck> send(SendMessageCommand command) {
        return Mono.fromCallable(() ->
            sendMessageTxService.sendInTransaction(command))
            .subscribeOn(jpaScheduler)
            .timeout(Duration.ofSeconds(3));
    }
}

```

11.4 事务服务写法

```

@Service
public class SendMessageTxService {

    private final SendMessageValidationChain validationChain;
    private final MessageRepository messageRepository;
    private final MessageSeqRepositoryCustom messageSeqRepository;
}

```

```

private final OutboxEventRepository outboxEventRepository;
private final IdGenerator idGenerator;

public SendMessageTxService(
    SendMessageValidationChain validationChain,
    MessageRepository messageRepository,
    MessageSeqRepositoryCustom messageSeqRepository,
    OutboxEventRepository outboxEventRepository,
    IdGenerator idGenerator
) {
    this.validationChain = validationChain;
    this.messageRepository = messageRepository;
    this.messageSeqRepository = messageSeqRepository;
    this.outboxEventRepository = outboxEventRepository;
    this.idGenerator = idGenerator;
}

@Transactional
public SendMessageAck sendInTransaction(SendMessageCommand command) {
    Optional<MessageEntity> existing = messageRepository
        .findBySenderIdAndClientMsgId(command.senderId(),
command.clientMsgId());

    if (existing.isPresent()) {
        return SendMessageAck.from(existing.get());
    }

    SendMessageContext context = SendMessageContext.from(command);
    validationChain.validate(context);

    long seq = messageSeqRepository.nextSeq(command.targetId());

    MessageEntity message = MessageEntity.create(
        idGenerator.nextId(),
        command.tenantId(),
        command.targetId(),
        command.threadId(),
        seq,
        command.senderId(),
        command.clientMsgId(),
        command.contentType(),
        command.content()
    );

    messageRepository.save(message);

    OutboxEventEntity event = OutboxEventEntity.messageCreated(
        idGenerator.nextId(),

```

```

        message
    );

    outboxEventRepository.save(event);

    return SendMessageAck.from(message);
}
}

```

12. 设计模式

12.1 Chain of Responsibility

用于消息发送前的校验链。

校验内容包括：

1. 用户登录状态。
2. target 是否存在。
3. target 是否可用。
4. 用户是否是成员。
5. 用户是否被禁言。
6. 频道是否允许普通成员发言。
7. 消息内容是否合法。
8. 是否触发限流。

接口：

```

public interface SendMessageValidator {

    int order();

    void validate(SendMessageContext context);
}

```

校验链：

```

@Component
public class SendMessageValidationChain {

    private final List<SendMessageValidator> validators;

    public SendMessageValidationChain(List<SendMessageValidator> validators) {

```

```

        this.validators = validators.stream()
            .sorted(Comparator.comparingInt(SendMessageValidator::order))
            .toList();
    }

    public void validate(SendMessageContext context) {
        for (SendMessageValidator validator : validators) {
            validator.validate(context);
        }
    }
}

```

12.2 Strategy

用于区分私聊、群聊、频道的不同投递策略。

```

public interface MessageDeliveryStrategy {

    boolean supports(TargetType targetType);

    DeliveryPlan buildPlan(MessageCreatedEvent event);
}

```

策略路由：

```

@Component
public class MessageDeliveryStrategyRouter {

    private final List<MessageDeliveryStrategy> strategies;

    public MessageDeliveryStrategyRouter(List<MessageDeliveryStrategy>
strategies) {
        this.strategies = strategies;
    }

    public MessageDeliveryStrategy route(TargetType targetType) {
        return strategies.stream()
            .filter(strategy -> strategy.supports(targetType))
            .findFirst()
            .orElseThrow(() -> new BizException("UNSUPPORTED_TARGET_TYPE"));
    }
}

```

12.3 Outbox Pattern

用于解决消息落库和投递事件之间的一致性问题。

核心原则：

同一个事务内：

1. 保存 message。
2. 保存 outbox_event。

事务提交后：

1. OutboxDispatcher 异步扫描事件。
2. 执行投递。
3. 成功后标记 SUCCESS。
4. 失败后重试。

12.4 Adapter Pattern

用于隔离外部设施。

例如 WebSocket 推送：

```
public interface ConnectionPushPort {  
  
    boolean pushToUser(Long userId, ServerMessage message);  
  
    boolean pushToConnection(String connectionId, ServerMessage message);  
}
```

单体实现：

```
@Component  
public class LocalConnectionPushAdapter implements ConnectionPushPort {  
  
    private final ConnectionManager connectionManager;  
  
    @Override  
    public boolean pushToUser(Long userId, ServerMessage message) {  
        return connectionManager.pushToUser(userId, message);  
    }  
  
    @Override  
    public boolean pushToConnection(String connectionId, ServerMessage message)  
    {
```

```

        return connectionManager.pushToConnection(connectionId, message);
    }
}

```

未来多节点实现：

```

@Component
public class RedisRouteConnectionPushAdapter implements ConnectionPushPort {

    @Override
    public boolean pushToUser(Long userId, ServerMessage message) {
        // 1. 查询 Redis 在线路由。
        // 2. 如果用户在本节点，直接推送。
        // 3. 如果用户在其他节点，通过 Redis Pub/Sub 或 MQ 转发。
        return true;
    }

    @Override
    public boolean pushToConnection(String connectionId, ServerMessage message)
    {
        return true;
    }
}

```

12.5 Factory Method

用于创建不同类型的 ChatTarget。

```

public interface ChatTargetFactory {

    ChatTargetCreateResult create(CreateChatTargetCommand command);
}

```

可以分别实现：

1. DirectChatFactory
2. GroupChatFactory
3. ChannelFactory

12.6 Template Method

用于统一消息发送流程。

固定流程：

1. 构建上下文。
2. 执行校验。
3. 分配 seq。
4. 创建消息。
5. 保存消息。
6. 写 Outbox。
7. 返回 ACK。

不同消息类型只覆盖部分扩展点。

13. 消息投递设计

13.1 投递模式

场景	模式	说明
私聊	写扩散	更新双方 UserInbox
小群	写扩散	更新所有成员 UserInbox
大群	读扩散	不更新所有成员，用户打开后拉取
频道	读扩散为主	广播消息不为全部订阅者写 inbox

13.2 小群写扩散

```
message_created event
  ↓
  查询成员
  ↓
  批量 upsert user_inbox
  ↓
  在线推送
  ↓
  标记 outbox success
```

13.3 大群读扩散

```
message_created event
  ↓
  只更新 message / target latest seq
  ↓
  推送在线活跃用户
```

↓
离线用户不写 inbox
↓
用户打开群时按 lastReadSeq 拉取

13.4 UserInbox upsert SQL

```
INSERT INTO im_user_inbox (  
    user_id,  
    target_id,  
    last_message_id,  
    last_seq,  
    last_read_seq,  
    unread_count,  
    pinned,  
    muted,  
    updated_at  
)  
VALUES (  
    :userId,  
    :targetId,  
    :messageId,  
    :seq,  
    0,  
    :unreadDelta,  
    false,  
    false,  
    now()  
)  
ON CONFLICT (user_id, target_id)  
DO UPDATE SET  
    last_message_id = EXCLUDED.last_message_id,  
    last_seq = EXCLUDED.last_seq,  
    unread_count = im_user_inbox.unread_count + EXCLUDED.unread_count,  
    updated_at = now();
```

14. WebSocket 连接管理

14.1 ConnectionManager

单体阶段使用本地内存维护连接。

```
user_id -> connection list
connection_id -> connection
```

示例结构：

```
@Component
public class ConnectionManager {

    private final Map<Long, Set<DeviceConnection>> userConnections = new
ConcurrentHashMap<>();
    private final Map<String, DeviceConnection> connectionMap = new
ConcurrentHashMap<>();

    public DeviceConnection register(Long userId, String deviceId, String
sessionId) {
        String connectionId = UUID.randomUUID().toString();

        DeviceConnection connection = new DeviceConnection(
            connectionId,
            userId,
            deviceId,
            sessionId
        );

        userConnections
            .computeIfAbsent(userId, key -> ConcurrentHashMap.newKeySet())
            .add(connection);

        connectionMap.put(connectionId, connection);

        return connection;
    }

    public void unregister(DeviceConnection connection) {
        Set<DeviceConnection> connections =
userConnections.get(connection.userId());
        if (connections != null) {
            connections.remove(connection);
        }

        connectionMap.remove(connection.connectionId());
        connection.close();
    }

    public boolean pushToUser(Long userId, ServerMessage message) {
        Set<DeviceConnection> connections = userConnections.get(userId);
```

```

        if (connections == null || connections.isEmpty()) {
            return false;
        }

        boolean pushed = false;
        for (DeviceConnection connection : connections) {
            pushed |= connection.tryEmit(message);
        }

        return pushed;
    }
}

```

14.2 每连接发送队列

```

public class DeviceConnection {

    private final String connectionId;
    private final Long userId;
    private final String deviceId;
    private final String sessionId;

    private final Sinks.Many<String> outboundSink;

    public DeviceConnection(
        String connectionId,
        Long userId,
        String deviceId,
        String sessionId
    ) {
        this.connectionId = connectionId;
        this.userId = userId;
        this.deviceId = deviceId;
        this.sessionId = sessionId;
        this.outboundSink = Sinks.many()
            .unicast()
            .onBackpressureBuffer();
    }

    public boolean tryEmit(ServerMessage message) {
        String json = Jsons.toJson(message);
        Sinks.EmitResult result = outboundSink.tryEmitNext(json);
        return result.isSuccess();
    }

    public Flux<String> outboundFlux() {

```

```

        return outboundSink.asFlux();
    }

    public void close() {
        outboundSink.tryEmitComplete();
    }

    public String connectionId() {
        return connectionId;
    }

    public Long userId() {
        return userId;
    }
}

```

14.3 慢连接处理

每个连接必须设置发送队列上限。

当队列满时：

消息类型	处理方式
普通聊天消息	断开连接，让客户端重连后按 seq 同步
系统通知	尽量保留，失败后可重试
typing 状态	直接丢弃
presence 状态	直接丢弃

原则：

WebSocket 推送是实时优化。
可靠性依赖 DB + seq + 客户端同步。

15. Outbox Dispatcher

15.1 拉取事件

使用 PostgreSQL `FOR UPDATE SKIP LOCKED` 实现多 worker 安全拉取。

```

SELECT id
FROM im_outbox_event
WHERE status = 'PENDING'
      AND (next_retry_at IS NULL OR next_retry_at <= now())
ORDER BY id ASC
FOR UPDATE SKIP LOCKED
LIMIT :limit;

```

15.2 Dispatcher 伪代码

```

@Component
public class MessageOutboxDispatcher {

    private final Scheduler jpaScheduler;
    private final OutboxEventRepositoryCustom outboxRepositoryCustom;
    private final OutboxEventTxService outboxEventTxService;
    private final MessageDeliveryService messageDeliveryService;

    @Scheduled(fixedDelayString = "${im.delivery.outbox-fixed-delay-ms:100}")
    public void dispatch() {
        Mono.fromCallable(() ->
outboxRepositoryCustom.fetchAndLockPendingEvents(100))
            .subscribeOn(jpaScheduler)
            .flatMapMany(Flux::fromIterable)
            .flatMap(event ->
                messageDeliveryService.deliver(event)
                    .then(Mono.fromRunnable(() ->
outboxEventTxService.markSuccess(event.getId()))
                        .subscribeOn(jpaScheduler))
                    .onErrorResume(ex ->
                        Mono.fromRunnable(() ->
outboxEventTxService.markRetry(event.getId(), ex))
                            .subscribeOn(jpaScheduler)
                        ),
                    8
                )
            .subscribe();
    }
}

```

15.3 重试策略

retry_count	next_retry_at
1	5 秒后

retry_count	next_retry_at
2	30 秒后
3	2 分钟后
4	10 分钟后
5	标记 FAILED

16. 并发设计

16.1 发送幂等

客户端每条消息生成唯一 `clientMsgId`。

服务端唯一约束：

```
UNIQUE (sender_id, client_msg_id)
```

如果客户端重试：

1. 服务端查询到已有消息。
2. 不重复插入。
3. 返回已有消息的 ACK。

16.2 消息顺序

每个 ChatTarget 单独维护 seq。

```
target_id = 10001  
seq = 1, 2, 3, 4, 5...
```

客户端同步：

```
GET /api/im/targets/10001/messages?afterSeq=100&limit=50
```

16.3 热点 target 优化

第一版直接使用 DB 自增 seq。

高并发热点群后续改为号段分配。

```
UPDATE im_chat_target_seq
SET current_seq = current_seq + 1000
WHERE target_id = :targetId
RETURNING current_seq - 999 AS start_seq,
          current_seq AS end_seq;
```

应用内使用 `AtomicLong` 分配。

16.4 限流

至少三层限流：

限流级别	示例
用户级	单用户每秒最多发送 10 条
target 级	单群每秒最多接收 200 条
全局级	当前节点每秒最多处理 N 条

单体第一版可以使用本地令牌桶。

未来多节点使用 Redis RateLimiter。

16.5 数据库连接池

JPA Scheduler 线程数不要远大于 Hikari 最大连接数。

推荐：

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 30

im:
  jpa:
    scheduler:
      thread-size: 32
```

16.6 消息表索引

核心查询：


```
SELECT *
FROM im_message
WHERE target_id = :targetId
      AND seq > :afterSeq
ORDER BY seq ASC
LIMIT :limit;
```

必须有索引：

```
CREATE INDEX idx_message_target_seq_desc
ON im_message (target_id, seq DESC);
```

17. 文件系统预留

17.1 当前阶段

当前只保存附件元数据。

消息内容引用 `attachmentId`。

```
{
  "contentType": "IMAGE",
  "content": {
    "attachmentId": 80001,
    "url": null,
    "width": 1080,
    "height": 720
  }
}
```

17.2 后续接入 MinIO

设计接口：

```
public interface FileStorageAdapter {

    UploadedFile upload(FileUploadCommand command);

    DownloadUrl generateDownloadUrl(Long fileId);
}
```

第一版：

```
public class LocalFileStorageAdapter implements FileStorageAdapter {  
}
```

后续：

```
public class MinioFileStorageAdapter implements FileStorageAdapter {  
}
```

18. 安全设计

18.1 WebSocket 鉴权

浏览器 WebSocket 不方便设置自定义 Header。

推荐使用短期 `ws_ticket`：

1. 客户端通过 HTTP API 获取 `ws_ticket`。
2. 客户端连接 WebSocket 时携带 `ticket`。
3. 服务端校验 `ticket`。
4. `ticket` 使用后失效或短时间过期。

连接示例：

```
ws://localhost:8080/ws/im?ticket=xxx
```

18.2 权限校验

发送消息前必须校验：

1. 用户是否登录。
2. `target` 是否存在。
3. `target` 是否被封禁。
4. 用户是否是成员。
5. 用户是否被禁言。
6. 频道是否允许当前用户发言。
7. 消息内容是否合法。

18.3 内容安全

第一版可以只做基础校验：

- 1. 内容长度限制。
- 2. contentType 白名单。
- 3. JSON schema 校验。
- 4. 文件大小限制。

后续扩展：

- 1. 敏感词。
- 2. 反垃圾消息。
- 3. 风控频率。
- 4. 图片审核。
- 5. 链接审核。

19. 可观测性设计

19.1 日志字段

关键日志需要包含：

字段	说明
traceId	请求链路 ID
userId	当前用户
targetId	消息目标
messageId	消息 ID
clientMsgId	客户端消息 ID
seq	target 内消息序号
op	操作类型
costMs	耗时
result	成功或失败

19.2 指标

建议暴露以下指标：

指标	说明
im_ws_connections	当前 WebSocket 连接数
im_message_send_total	消息发送总数
im_message_send_failed_total	消息发送失败数
im_outbox_pending_count	待投递事件数量
im_outbox_retry_count	重试事件数量
im_delivery_success_total	投递成功数
im_delivery_failed_total	投递失败数
im_connection_slow_total	慢连接数量
im_inbound_qps	进站 QPS
im_outbound_qps	出站 QPS

19.3 告警

需要告警的情况：

1. Outbox pending 数量持续增长。
2. 消息发送失败率升高。
3. JPA 线程池排队严重。
4. Hikari 连接池耗尽。
5. WebSocket 连接数异常下降。
6. 投递失败率升高。
7. 慢连接数量异常。

20. 配置示例

```
server:
  port: 8080

spring:
  application:
    name: im-monolith

datasource:
  url: jdbc:postgresql://localhost:5432/im
  username: im
  password: im
  hikari:
```

```
    pool-name: im-hikari-pool
    maximum-pool-size: 30
    minimum-idle: 5
    connection-timeout: 3000
    idle-timeout: 600000
    max-lifetime: 1800000

jpa:
  open-in-view: false
  hibernate:
    ddl-auto: validate
  properties:
    hibernate:
      format_sql: false
      jdbc:
        batch_size: 100
        order_inserts: true
        order_updates: true

flyway:
  enabled: true
  locations: classpath:db

im:
  jpa:
    scheduler:
      thread-size: 32
      queue-size: 10000

websocket:
  max-frame-size: 65536
  outbound-queue-size: 500
  heartbeat-interval-seconds: 30

delivery:
  outbox-batch-size: 100
  outbox-fixed-delay-ms: 100
  small-group-threshold: 500

rate-limit:
  user-send-per-second: 10
  target-send-per-second: 200
```

21. 推荐开发顺序

阶段一：基础模型

完成：

1. ChatTarget
2. DirectChat
3. GroupChat
4. Channel
5. TargetMember
6. Message
7. MessageSeq
8. UserInbox
9. OutboxEvent

产出：

1. Flyway DDL。
2. JPA Entity。
3. Repository。
4. 基础单元测试。

阶段二：HTTP 消息发送

先实现 HTTP API：

```
POST /api/im/messages
```

完成：

1. 幂等。
2. 权限校验。
3. seq 生成。
4. message 落库。
5. outbox 落库。
6. 返回 SendAck。

阶段三：消息拉取和会话列表

实现：

```
GET /api/im/inboxes
```

```
GET /api/im/targets/{targetId}/messages?afterSeq=xxx
```

```
GET /api/im/targets/{targetId}/messages?beforeSeq=xxx
POST /api/im/targets/{targetId}/read
```

此时即使没有 WebSocket，也已经是可用的离线 IM。

阶段四：Outbox Dispatcher

实现：

1. 扫描 outbox。
2. 更新 inbox。
3. 标记 success。
4. 失败重试。
5. 死信处理。

阶段五：WebSocket

实现：

1. WebSocket 鉴权。
2. ConnectionManager。
3. 心跳。
4. SEND_MESSAGE。
5. SEND_ACK。
6. MESSAGE_PUSH。
7. DELIVERY_ACK。
8. READ_TARGET。

阶段六：并发保护

补齐：

1. 用户级限流。
2. target 级限流。
3. 全局限流。
4. 慢连接踢出。
5. outbound queue 上限。
6. Outbox 批量处理。
7. 指标和告警。

阶段七：扩展能力

逐步实现：

1. Redis 在线路由。
2. 多节点 WebSocket。

3. MQ 替代本地 Outbox Dispatcher。
4. MinIO 文件系统。
5. 消息搜索。
6. 风控和审核。
7. 多租户隔离增强。

22. 后续微服务演进

当前模块化单体可以平滑拆成：

```
im-gateway-service
    WebSocket 长连接、连接路由、协议解析

im-message-service
    消息发送、消息存储、seq 生成

im-delivery-service
    投递、离线同步、已读回执

im-target-service
    私聊、群聊、频道、成员关系

im-file-service
    文件上传、下载、MinIO

im-admin-service
    管理后台、风控、审计
```

演进时需要新增：

1. Redis 在线路由。
 2. MQ 消息总线。
 3. 分布式限流。
 4. 分布式 ID。
 5. 统一认证服务。
 6. OpenTelemetry 链路追踪。
-

23. 关键取舍

23.1 为什么不用 Conversation 作为核心实体？

因为 Conversation 容易混淆两个概念：

1. 消息可以发到哪里。
2. 用户会话列表展示什么。

本设计中：

ChatTarget = 消息投递目标
UserInbox = 用户会话列表投影

这样私聊、群聊、频道、thread 的关系更清晰。

23.2 为什么使用 Outbox？

因为消息落库和消息投递不是同一个事务边界。

如果在发送消息事务内直接 WebSocket 推送，会出现：

1. DB 回滚但消息已推送。
2. 推送失败导致发送失败。
3. 慢连接拖垮发送链路。
4. 投递副作用无法可靠重试。

Outbox 可以保证：

消息落库成功，则事件一定存在。
事件存在，则可以异步重试。

23.3 为什么大群用读扩散？

如果万人群每发一条消息就更新一万条 inbox，数据库压力非常大。

大群读扩散的原则是：

消息只写一次。
用户打开时再拉。

这牺牲了部分精确未读，但换来系统可扩展性。

23.4 为什么 WebSocket 推送不作为可靠性唯一来源？

WebSocket 是长连接实时通道，不稳定因素很多：

1. 弱网。
2. 断线。
3. 慢连接。
4. 浏览器刷新。
5. 移动端后台冻结。

可靠性应该由：

```
DB message + target seq + afterSeq sync
```

保证。

24. 当前版本最小闭环

第一版只需要完成：

1. Spring Boot WebFlux 单体。
2. PostgreSQL + JPA。
3. ChatTarget 统一消息目标。
4. DirectChat / GroupChat / Channel。
5. Message 落库。
6. target_id + seq。
7. clientMsgId 幂等。
8. OutboxEvent。
9. UserInbox。
10. HTTP 拉消息。
11. HTTP 会话列表。
12. WebSocket 推送。
13. 断线后 afterSeq 同步。

做到这一步，系统已经具备一个 IM 内核的基本形态。

25. 总结

本设计采用：

模块化单体

- + Spring Boot WebFlux
- + Spring Data JPA
- + PostgreSQL
- + ChatTarget 统一消息目标
- + UserInbox 会话列表投影
- + Outbox 异步可靠投递
- + target 内 seq 顺序
- + 小群写扩散
- + 大群读扩散

核心思想是：

发送链路短。

投递链路异步。

消息可靠性靠 DB。

实时性靠 WebSocket。

扩展性靠模块边界。

当前阶段不急着拆微服务。先把领域模型、消息链路、并发边界做对。后面规模上来，再把模块平移成服务。这样不是推倒重来，而是顺势演进。